

PRAVAAH

PROBLEM STATEMENT Intelligent Public Transport Crowding & Route Prediction



Forecast how crowded and delayed a bus will be **when it reaches you**, then decide the better route or departure time **before the problem happens**.



Built on open GTFS and GTFS-Realtime feeds: real network, real vehicle movement, real operator-reported crowding.

TEAM

Rushil Jain 241EC148

Sumit 241CV150

Mayank Tiwari 241CV235

Somyak Mohanta 241CS257

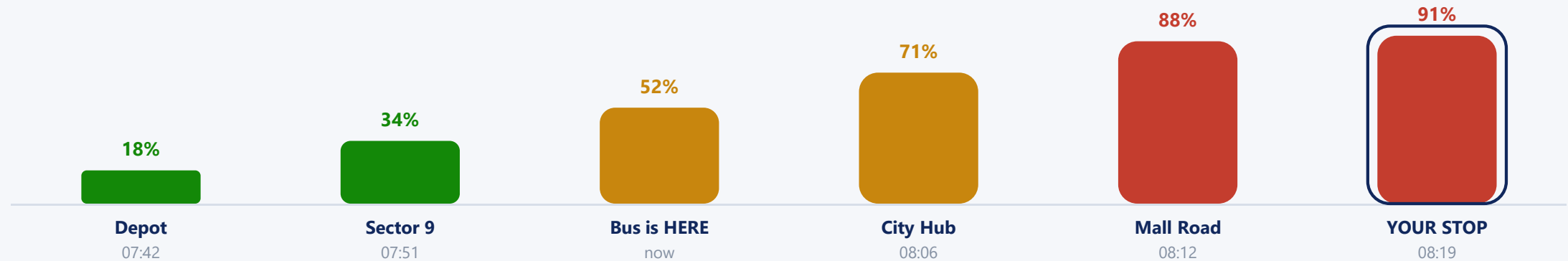
Krupa Marol 241CV231

Rajvardhan 241MT041

A bus that looks empty now can be full when it reaches you

Current occupancy is the wrong signal. What matters is the load at your stop, at your boarding time.

Predicted occupancy of one bus along its route the passenger boards at the far end



PASSENGER

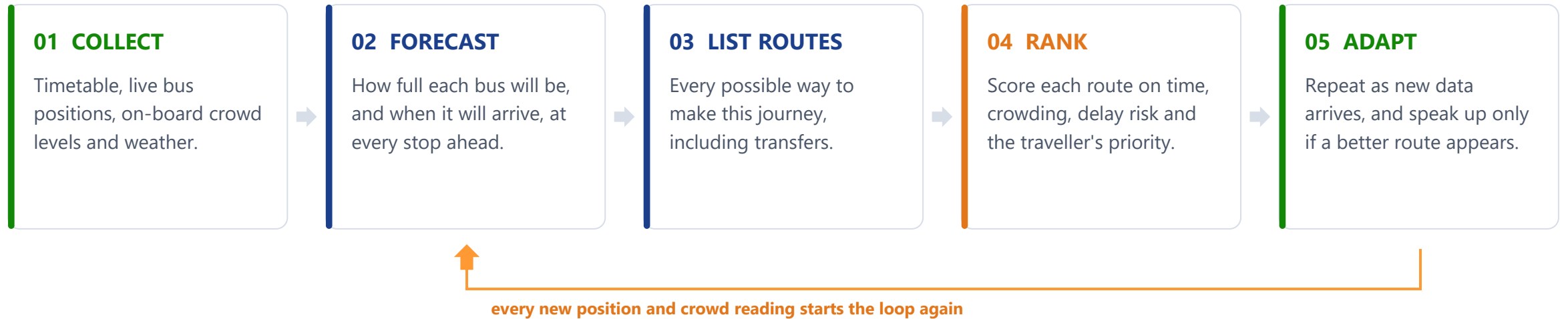
- Route apps optimise for time, not for whether you can board.
- No app answers “should I just leave 15 minutes later?”
- A 30-min trip that is sometimes 50 looks identical to one that never varies.

OPERATOR

- Control rooms see overcrowding only after the corridor has failed.
- Adding a bus or changing headway is reactive, never quantified first.
- No continuous record of where demand actually exceeded capacity.

A prediction-to-decision loop, not another tracking map

Routing will stay deterministic and explainable. Machine learning predicts the conditions those routes will face.



Where the intelligence sits Steps 1, 3 and 5 are engineering. Step 2 is the learning: predicting how crowding and delay build up along a route and across the hours of the day, and stating how confident it is in each figure. Step 4 then applies a scoring rule a person can read, which is why every recommendation can explain itself.

The same trip, three honest answers

Every option shows predicted crowd at your stop, delay risk and a plain-language reason.

Option	Travel	Crowd at YOUR stop	Delay risk	Verdict
Bus 14	34 min	91% very crowded	High	Avoid
Bus 31	31 min	72% standing	Medium	Fastest
Bus 22	39 min	42% seats likely	Low	Recommended

The traveller sets the priority. The table above re-orders to match it.

Fastest

Least crowded

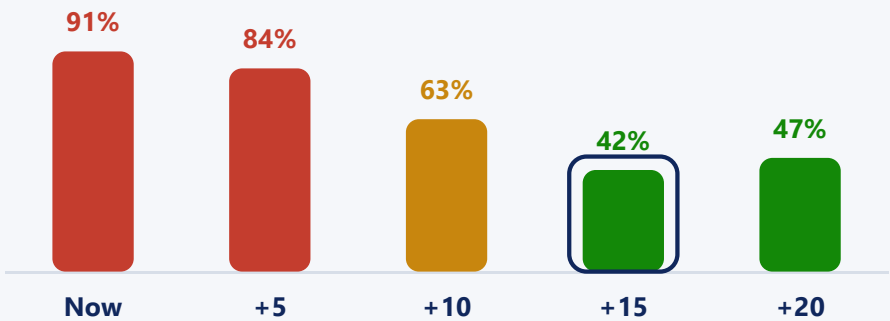
Most reliable

Balanced

selected

Uncertainty is shown, not hidden. Occupancy p10 0.68 / p50 0.84 / p90 0.95, arrival 17:31 to 17:39. Missing occupancy shows as **unknown**, never as empty.

The advice no other app gives: “leave 15 minutes later”



Same arrival window, 42% instead of 91% full. We suggest waiting only when the gain clears a threshold, and re-route mid-journey only when the better option actually flips.

What existing products do not do

The differentiator is forecasting at the decision point, plus an operator product no consumer app will build.

	Live vehicle map	Crowd forecast at your stop	Uncertainty / reliability	Advice to travel a little later	Operator decision support
Google Maps / Transit	●	◐	○	○	○
Chalo, Tummoc & similar	●	◐	○	○	○
PRAVAAH	●	●	●	●	●

● full ◐ partial / route-level average ○ not offered

Predictive control room

Forecast hotspots with lead time, and score interventions before they are ordered. No consumer app wants this relationship.

Your stop, your time

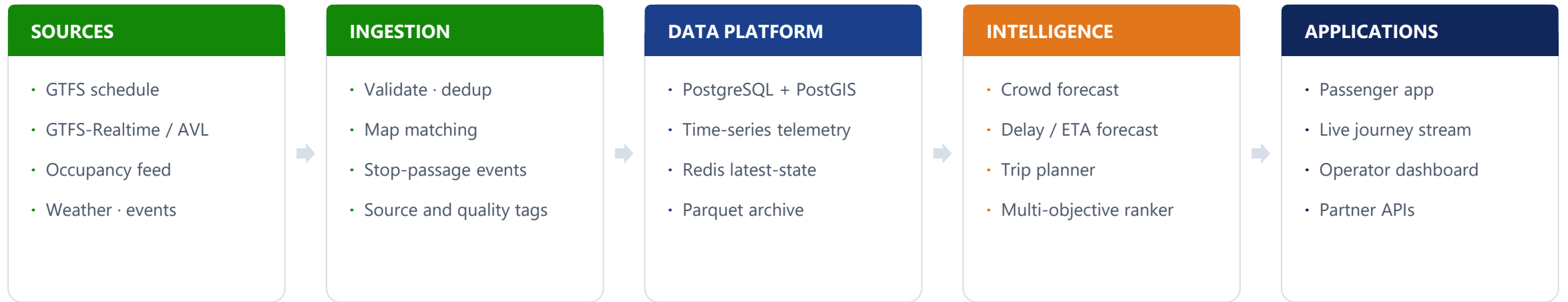
Not a route-level historical average. It is the load of that vehicle when it reaches you, propagated stop by stop.

Spreading the peak

Telling one traveller to leave ten minutes later takes one body out of the fullest bus. Applied across a city, the peak itself flattens, and no new vehicle was bought to do it.

System architecture: five planes, stable contracts

Adapters isolate each city's feed. Everything downstream uses one canonical schema, so a new city is a simple config change.



EVERY READING IS KEPT, NOTHING IS OVERWRITTEN

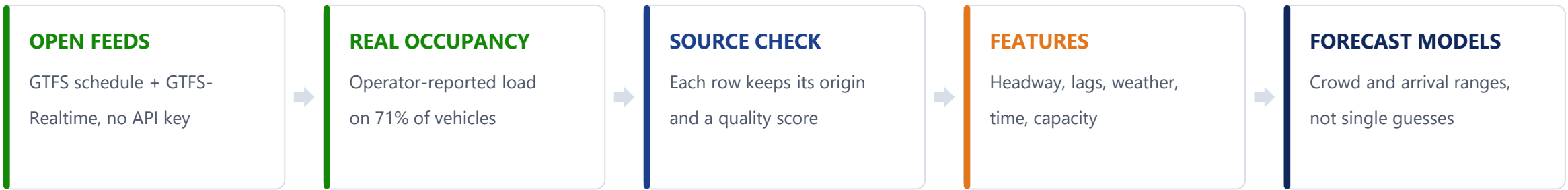
Each reading is stored once, with its time and its source. Because the record stays complete, any past hour can be replayed exactly: the demonstration repeats, an earlier recommendation can be checked, and the training set grows on its own.

THE PROTOTYPE AND THE FULL SYSTEM SHARE INTERFACES

The prototype runs every service as one application with background workers. Scaling up moves them onto separate machines, but the data formats and the interfaces between them stay exactly as they are.

All model inputs are drawn from live operational data

Network, movement, crowding, delay and weather, every one of them taken from a live public feed.



Input	Source	Status
Network topology	GTFS Schedule: 399 routes, 10,297 stops, 2.2M stop-times	Measured
Vehicle movement	GTFS-Realtime VehiclePositions, ~560 vehicles every 20 s	Measured
Crowding labels	Operator-reported occupancy_status on ~71% of vehicles	Measured
Delay labels	TripUpdates arrival delay, plus observed stop passages	Measured
Weather	Open-Meteo hourly history and forecast	Measured

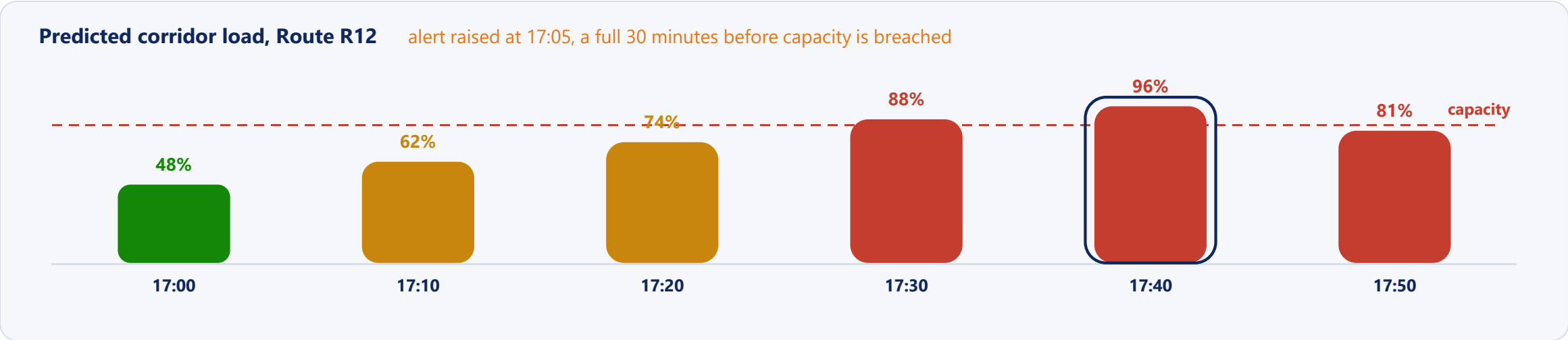
COLLECTION IS ALREADY RUNNING

2 million
vehicle readings recorded across the network

1.4 million
carrying a measured occupancy value

From reacting to overcrowding, to preventing it

The capability consumer navigation products do not offer, and the one a transport authority is most likely to procure.



PREDICTED HOTSPOTS

Exceedance probability, duration and lead time. Escalated only when the lower confidence bound breaches.

SERVICE INSTABILITY

Headway tracked directly: bunching, recurrent delay segments, fleet state with data-freshness flags.

SCENARIO EVALUATION

“Add one bus at 17:20”: predicted load relief, compared against the option of doing nothing.

Technology stack and delivered capabilities

TECHNOLOGY

Backend Python FastAPI with asynchronous feed workers

Models LightGBM quantile regression, scikit-learn

Routing Established GTFS routing library (RAPTOR/CSA)

Database PostgreSQL with PostGIS for the network graph

State Redis for live vehicle state, Parquet for history

Interface React with MapLibre

Deployment Docker Compose, single-command startup

DELIVERABLES

- Journey planner returning ranked routes with predicted crowding, arrival time and delay risk.
- Four preference profiles the traveller can switch between.
- Departure-time advice when travelling later gives a materially better journey.
- Live re-scoring that updates an active journey as conditions change.
- Operator dashboard showing predicted hotspots with lead time.
- Scenario tool comparing an added vehicle or a headway change against leaving the service unchanged.
- Replayable archive of every reading, for retraining and audit.

Who gains, and by how much

Benefits across the three groups the system serves, with the criteria each claim is held to.

PASSENGERS

9 min

saved for every full bus avoided

3 in 4

peak journeys have a quieter option within 15 minutes

Equity

greatest gain for shift workers, students and elderly riders

OPERATORS

30 min

of warning before a corridor exceeds capacity

8 to 12%

peak load removed when 5% of riders travel later

No new hardware

the system runs on schedule and location data already held

THE CITY

1 server

runs a 500-vehicle fleet, with no licence cost

Any GTFS city

adopts the system without rewriting it

Riders retained

comfort and reliability are why they leave for cars

HOW THE SYSTEM IS TESTED

- Crowd forecasts use a chronological hold-out, never a random split.
- Arrival times must beat the published timetable and the historical median.
- Calibration: a 90% interval must contain the outcome 90% of the time.
- Journey search completes within 2.5 seconds at the 95th percentile.

REFERENCES

GTFS Schedule & GTFS-Realtime specifications (gtfs.org) · MBTA open GTFS-RT feed, cdn.mbta.com, no key required · Delhi Open Transit Data, otd.delhi.gov.in · Transport for London BUSTO open dataset · Open-Meteo historical weather API.